Deployment and Setup Guide - Addis Ababa Bus Tracking System

1. Overview

The system runs in two ways: on one computer for development, or online for real use.

Part	Folder	Local address	Online home
Database	dockercompose.yml	port 5434 (Docker)	Neon (PostgreSQL with PostGIS)
Backend	backend	http://localhost:3000	Render (Docker service)
Rider app	user-app	http://localhost:5174	https://addis-ababa-bus-tracking-system.vercel.app/
Driver app	driver-app	http://localhost:5173	Vercel
Admin app	admin-app	http://localhost:5175	Vercel

2. Requirements

Software on the computer

- Node.js 22 or newer, with npm
- Docker Desktop, for the local database
- Git
- phone through a temporary https
- address

Accounts for the online copy

- GitHub, for the code
- Neon, for the database
- Render, for the backend
- Vercel, for the three apps

3. Local setup

1. **Create backend/.env** with the values in section 4. Never commit this file.
2. **Start the database.** From the project root:
   ```
   docker compose --env-file backend/.env up -d db
   ```
3. **Prepare the backend.** In the `backend` folder:

   ```
   npm install
   npm run db:migrate
   npm run bootstrap
   npm run dev
   ```

   `db:migrate` creates the tables. `bootstrap` loads routes 49 and 61 and creates the first admin account from `ADMIN_USERNAME` and `ADMIN_PASSWORD`. It skips anything that already exists, so it is safe to run again. The server listens on port 3000.
4. **Start each app.** In each of `user-app`, `driver-app` and `admin-app`, create a `.env` file with one line, `VITE_API_URL=http://localhost:3000`, then run:

   ```
   npm install
   npm run dev
   ```

4. Testing with the fake bus and a phone

Fake bus. A script drives a bus along a real route, so the system can be tested without a real bus. In backend :

npx tsx src/scripts/createshift.ts bus1 61 "toward Legehar" AA-1234 npx tsx src/scripts/fake-bus.ts --send --live --user bus1 --pass <password>

createshift.ts takes a driver username, a route number, a direction name and a plate. -send sends the points. Without it the script only prints them. --live runs in real time. Do not use --delay for live tests, because it makes the points look old. Ctrl+C stops the bus, which is also the way to test the no-signal behaviour.

Option	Effect
`--stop-at 40`	The bus stops at 40% of the road and holds until Ctrl+C

`--wait-min 3 --stay`	The bus waits 3 minutes at the terminal and does not leave

Fake bus on the online copy. In PowerShell, point the script at the online backend. Close the window afterwards.

```
$env:API_URL="https://<service>.onrender.com" npx tsx src/scripts/fake-bus.ts --send --live --user bus1 --pass <password>
```

The driver, the vehicle and the shift must exist in the online database first. Create them in the admin app.

Phone, local copy. A phone needs https for GPS. Start a tunnel for the backend (`cloudflared tunnel --url http://localhost:3000`), put its address in `VITE_API_URL` , restart the app, and open the app's own tunnel address on the phone.